SpiceFunctions

```
import numpy as np
from PyLTSpice import RawRead
import os
import re
import glob
from PyLTSpice import SimCommander
import subprocess


class SpiceEvaluator:
    def __init__(self, netlist_path, ltspice_path=None):
        self.netlist_path = os.path.abspath(netlist_path)
        self.ltspice_path = ltspice_path  # <-- Store the passed path
        self.params = self._discover_params()
        self.measures = self._discover_measures()
        print(f"[+] SpiceEvaluator initialized for:
{os.path.basename(netlist_path)}")

    def _discover_params(self):
        found_params = []
        with open(self.netlist_path, 'r', encoding='utf-8',
errors='ignore') as f:
            for line in f:
                if line.strip().upper().startswith('.PARAM'):
                    # Finds Rd in '.param Rd = 25000'
                    match = re.search(r'\.param\s+([a-zA-Z_]\w*)', line,
re.IGNORECASE)
                    if match:
                        found_params.append(match.group(1))
        return sorted(list(set(found_params)))

    def _discover_measures(self):
        found_measures = []
        with open(self.netlist_path, 'r', encoding='utf-8',
```

```
errors='ignore') as f:
            for line in f:
                if line.strip().upper().startswith('.MEAS'):
                    tokens = line.strip().split()
                    if len(tokens) >= 3:
                        found_measures.append((tokens[2] if
tokens[1].upper() in {'AC','DC','OP','TRAN','TF','NOISE'} else
tokens[1]).upper())
        return sorted(list(set(found_measures)))

    def _scrape_log(self, log_path):
        telemetry = {m: None for m in self.measures}
        num_pattern = r'([+-]?\d+(?:\.\d+)?(?:[eE][+-]?\d+)?)'

        with open(log_path, 'r', encoding='utf-8', errors='ignore') as f:
            lines = f.readlines()
            for m in self.measures:
                for line in lines:
                    if m.lower() in line.lower():
                        parts = re.split(r'[:=]|AT\s+', line,
flags=re.IGNORECASE)
                        if len(parts) > 1:
                            target_segment = parts[-1]
                            match = re.search(num_pattern,
target_segment)
                            if match:
                                telemetry[m] = float(match.group(1))
                                break
        return telemetry

    def __call__(self, **kwargs):
        # 1. Update netlist with Regex to handle any whitespace
        with open(self.netlist_path, 'r', encoding='utf-8') as f:
            lines = f.readlines()

        new_lines = []
        for line in lines:
            new_line = line
            # Check if this line is a .param line and if it matches one
```

```
of our updates
            for p_name, p_val in kwargs.items():
                # Matches ".param Rd", then optional spaces, '=', then
optional spaces
                pattern = rf'^\s*\.param\s+{re.escape(p_name)}\s*=\s*'
                if re.search(pattern, line, re.IGNORECASE):
                    new_line = f".param {p_name}={p_val}\n"
                    print(f"[*] Updated {p_name} to {p_val}")
            new_lines.append(new_line)

        with open(self.netlist_path, 'w', encoding='utf-8') as f:
            f.writelines(new_lines)

        # 2. Execute using the dynamic path parameter
        exe_path = self.ltspice_path if self.ltspice_path else
"LTspice.exe"
        cmd = [exe_path, "-b", "-run", self.netlist_path]

        result = subprocess.run(
            cmd,
            cwd=os.path.dirname(self.netlist_path),
            capture_output=True,
            text=True
        )

        if result.returncode != 0:
            raise RuntimeError(f"LTspice failed to execute. Check your
netlist syntax.\n{result.stderr}")

        # 3. Scrape
        log_path = os.path.splitext(self.netlist_path)[0] + ".log"
        return self._scrape_log(log_path)


class SpiceWaveformReader:
    """Reads LTspice binary .raw files and extracts clean NumPy frequency
arrays."""
    def __init__(self, raw_path):
        self.raw_path = raw_path
```

```
        self.raw = RawRead(raw_path)

    def get_ac_response(self, output_node="V(vout)"):
        """
        Returns full 1D NumPy arrays: (frequencies, gain_db, phase_deg)
        """
        # 1. Extract the frequency axis array
        freq = self.raw.get_trace("frequency").get_wave(0)

        # 2. Extract complex voltage array at your output node
        vout_complex = self.raw.get_trace(output_node).get_wave(0)

        # 3. Convert complex array into clean Magnitude (dB) and
Unwrapped Phase (Deg)
        gain_db = 20 * np.log10(np.abs(vout_complex))
        phase_deg = np.degrees(np.unwrap(np.angle(vout_complex)))

        return freq, gain_db, phase_deg
```